Rivet: Windows port status

Windows port of the macOS utilities app · branch `windows-port`, folder `windows/` · 9 October 2026

27	2,241	0 / 0	15
module areas, all 44 catalog features wired	tests passing (Core 1,767 · Imaging 128 · App 346)	build errors / warnings, both targets	languages, Windows strings included

> **Read this first.** None of the Windows-specific code has run on a Windows PC yet. Everything was built and verified on macOS: the Windows target (`net10.0-windows10.0.22621.0`) compiles, so every Win32/WinRT call is type-checked, and the app itself runs on a cross-platform target with fake platform services, which is what the tests and UI snapshots exercise. The next step is a pass on real Windows 10 22H2 and Windows 11 hardware using the manual test checklist in each `docs/modules/*.md`.

1. Where things stand

- **Stack:** C# / .NET 10, Avalonia 12, SkiaSharp, CsWin32, Vortice (D3D11/DXGI/Media Foundation), NAudio. The working name is **Rivet**: the project's trademark rules keep forks from using the original name, icon or update feed.
- **Code:** about 171,000 lines of source and 30,000 lines of tests in `windows/`: 5 projects (Core, Imaging, Platform.Windows, Platform.Fake, App) and 3 test projects.
- **Verified here:** `dotnet build Rivet.slnx` is clean, all 2,241 tests pass and `--selftest` prints `SELFTEST OK`. About 360 headless UI snapshots (light and dark, plus German, Russian and Japanese) were rendered and reviewed by eye.
- **Nothing is committed.** All work is untracked on `windows-port`. The CI workflow (`.github/workflows/windows.yml`: build, test, selftest, publish, Inno Setup installer) is written but not pushed.

2. What is in the app

Area	Features
Capture	Screenshot (area, window, full screen, scrolling), chooser with loupe and hint bar, preview, pins, QR/barcode reading, copy text from screen (OCR), colour picker, Recent captures history
Screenshot editor	Annotations, backdrops and presets shared with the recording editor, watermark, export, pin to screen
Screen recording	Display, window or area; system audio and microphone; countdown; pointer and typing tracks
Recording editor	Timeline, automatic zoom, looks, backdrop, cuts, H.264 and GIF export
System monitor	CPU (total and per core), GPU, memory pressure, disks, network with speed test, battery and power, temperatures where Windows exposes them, USB devices, process lists with Force Kill, alerts, tray tooltip readouts, always-on-top mini monitor
Energy and display	Keep awake (durations, automation, closed-lid mode with crash-safe restore), display brightness (laptop panel, DDC/CI, overlay dimming) with on-screen display
Maintenance	Cleaner (Recycle Bin only, `.reg` backups, schedule), Uninstaller with leftover scan, App updates (winget, Store, app feeds), Package manager (winget), Kill Process, Port Manager

Clipboard and text	Clipboard history (honours password-manager exclusion formats), paste as plain text, URL cleaner, text snippets
Launcher	Command Bar (apps, files, Windows Settings pages, calculator, units, dates, colours, emoji, snippets, links), Quick panel, Quick toggles
Sound	Volume mixer, output switcher, audio device priority, microphone mute, precise volume, headphone guard
Mouse and keyboard	Smooth scrolling, scroll inverter, mouse-button shortcuts, click and key debounce, Super key, quit protection, radial menu
Everyday tools	Shelf (drag stash), Scratchpad, Cleaning mode, Camera preview, Media tools (convert images, video and GIF; OCR)
AI agent usage	Local-only usage readout for Claude Code, Codex, OpenCode and Copilot CLI
App shell	Tray icon and panel, Settings window, onboarding, global shortcuts, toasts, single instance, settings backup and restore, updates through GitHub releases

3. What this session did

The previous session ran out of usage after merging six module areas. At that point four build agents were cut off mid-module. This session:

- **Resumed and finished the four interrupted areas** in their existing work copies: system monitor / keep awake / displays; cleaner / uninstaller / app updates / package manager / processes; clipboard / snippets / Command Bar / Quick panel / Quick toggles; shelf / radial menu / scratchpad / cleaning mode / camera / media tools. Then merged each into the main tree.
- **Fixed integration problems the merges exposed**, among them:
  - A namespace clash that broke the Windows build.
  - Radial menu quick-toggle slices that could never run, because of an action id mismatch.
  - Radial wheels on mouse side buttons, which would have lost those buttons to mouse-button shortcuts.
  - Panel section order not matching the spec.
  - The tray icon redrawing on every tooltip change.
  - A shortcut row that stopped updating when a Settings page was shown again.
  - Tests that assumed other modules were absent, including one whose failure broke seven unrelated recorder tests.
- **Cleaned up shared code:** removed four local copies of the shortcut row in favour of the shared one, and added on/off state to actions so Quick panel tiles light up.
- **Translated every Windows-only string** into the 14 other languages: about 850 keys × 14, in 378 new string files, all following Microsoft's Windows terminology for each language.
- **Added guard tests:** every translation keeps its placeholders, no Windows string mentions the Mac, Finder or ⌘, and the UI is rendered in German, Russian and Japanese. Ten macOS strings that still said "Mac" or "Finder" in some language now have Windows wording.

4. Left out on purpose

- **Dynamic Island / notch** surfaces: no Windows equivalent. Their data shows in the panel and in toasts instead.
- **Temporary share links** and the feedback endpoint: both use the original maintainer's server, which a fork must not use.

- **Window snapping**, side-button navigation, linear scrolling and pointer-acceleration toggles. Snapping is left to Windows' own feature.
- **Fan control** (needs low-level hardware access) and **CPU temperature** on most PCs: there is no driver-free API, and the app never installs drivers. The panel explains this instead of showing a number.
- **Per-app network traffic and SMART disk health**: both need administrator rights.
- **Display on/off, keyboard backlight, volume boost above 100 %**: unsafe or not possible without vendor drivers.
- **No elevated helper process.** Admin-only actions offer "Restart as administrator" instead, to avoid a second elevated entry point.
- **winget is driven through its command line**, not its COM API: the app is unpackaged. Revisit if it ever ships as MSIX.

Each module doc lists its smaller gaps under "Not implemented" and "Deviations".

5. Open items

Shared-code requests still open (details in the module docs)

- Tray menu submenus, and a replacement tray glyph for Keep awake
- An icon service for executables (process and app lists use a generic icon today)
- Session and power events (lock, sleep, resume) as one shared service
- A shared Raw Input service
- A `--release-modifiers` guard helper for the Super key
- Virtual-file drops (Outlook attachments) for the Shelf
- A selected-text service
- The tray icon rectangle, so the Shelf pill can dock to it
- A content-window registry, so editor windows are left out of recordings
- `IShelfIntake.Remove`
- Extra clipboard formats on `IClipboardService`
- An owner for `IRecordingEditor.OpenAsync`
- Merge the duplicate pin and QR windows of capture and the screenshot editor
- An accessory slot on Controls-tab switches
- Live caption and state on Controls rows

Things only a Windows machine can confirm

- Low-level keyboard and mouse hooks: timeouts, elevated windows, and the watchdog that reinstalls hooks Windows drops.
- Capture and recording on mixed-DPI and HDR setups.
- Hardware readings: DDC/CI and WMI brightness, battery IOCTLs, D3DKMT GPU temperature.
- Recycle Bin edge cases.
- Parsing of real winget output.
- Snippet expansion across IMEs.

Translations to review

The new strings were machine-translated with each language's macOS catalog as the glossary. Native speakers should review them, in particular these:

- Windows 11 Settings page names quoted in the Command Bar and in captions.
- New terms with no glossary entry: "readouts", "mini monitor", "pill".
- Microsoft Store product names.

6. Decisions for you

1. **Commit and push.** Nothing has been committed or pushed. Say the word and I'll commit the port on `windows-port` and open a PR that runs the Windows CI workflow, which is the first run on real Windows.
2. **Product name.** "Rivet" is a placeholder. It's set in one place: `Directory.Build.props`, or `tools/rename-product.sh`.
3. **Dynamic Island.** It was deferred. A Windows take, such as a slim top-of-screen strip, is possible as a separate project.
4. **Packaging.** The app ships unpackaged (Inno Setup, per-user install). MSIX would unlock winget's COM API and Store-grade update handling, but it changes how the app is installed.

7. Build and test

Task	Command (run in `windows/`)
Build both targets	`dotnet build Rivet.slnx`
Run all tests	`dotnet test Rivet.slnx` (on macOS, export `DOTNET_ROOT` first)
UI snapshots	written to `tests/artifacts/snapshots/`
Smoke test	`Rivet --selftest`
Module rules and contracts	`docs/MODULE_GUIDE.md`, `docs/modules/*.md`, `docs/specs/*.md`